University of Jeddah

Mobile Programming Project

(CCSW 431)

Fall 2025

Qanouni: Mobile Legal Platform

Authors:

Faisal Saud Alharthi 2341120

Abdulmalik Alzahrani 2340534

Abdullah AlMazroui 2340644

Omar abdulziz 2142901

Instructor:
Dr. Mohammad D. Alahmadi

Abstract

Qanouni is a mobile application that connects clients with verified lawyers and legal entities in Saudi Arabia. The app focuses on streamlining the legal request lifecycle: from submitting a detailed request with attachments, to lawyer acceptance, case creation, chat communication, scheduling, payments, and final rating. The system uses Firebase Authentication, Realtime Database, Firestore, and Firebase Storage to provide secure authentication, structured data, real-time messaging, and file handling. This document describes the problem addressed, the main stakeholders, user stories, sprints, functional requirements, user interface design, backend logic, database design, test cases, and future work.

Contents

1 Introduction **4**

2 Problem Statement & Objectives **4**
2.1 Problem Statement 4
2.2 Project Objectives 4

3 Stakeholders **4**
3.1 Stakeholder Roles 4

4 User Stories **5**
4.1 List of User Stories 5

5 Sprints **5**
5.1 Sprint 1 5
5.1.1 Progress Summary 5
5.1.2 Issues Faced 6
5.2 Sprint 2 6
5.2.1 Progress Summary 6
5.2.2 Issues Faced 6
5.3 Sprint 3 6
5.3.1 Progress Summary 7
5.3.2 Issues Faced 7

6 Functional Requirements **7**

7 User Interface (UI) Design **7**
7.1 Authentication Screen 8
7.2 Client home and notification 9
7.3 Lawyer Schedule Screen 10
7.4 Lawyer Chat Screen 11
7.5 Lawyer Home Dashboard 12
7.6 Lawyer Requests and Existing Chats 13
7.7 Lawyer Files and Profile Screens 14
7.8 Client Lawyer-Search and Request Dialog 15
7.9 Client Conversations and Chat Screen 16
7.10 Payment and Rating Screens 17

8 Backend Logic & Functionality **18**
8.1 Key Functions Overview 18
8.1.1 `register()` – New User Registration 18
8.1.2 `login()` – User Login 20
8.1.3 `submitRequest()` – Client Submits Request 21
8.1.4 `acceptRequest()` – Lawyer Accepts Request 23

8.1.5 `setupSpinner()` – Update Case Status from Chat 25
8.1.6 `saveRating()` – Save Lawyer Rating 26
8.1.7 `showAddScheduleDialog()` – Add New Schedule Slot 28

9 Database Design **30**
9.1 Collections / Nodes Overview 30
9.2 Node: users 31

10 Test Cases **33**

11 Challenges and Solutions **33**
11.1 Challenges Faced 33
11.2 Solutions Implemented 34

12 Conclusion & Future Work **34**
12.1 Conclusion 34
12.2 Future Work 34

13 Source Code **34**
13.1 GitHub Link 34
13.2 Figma Link 35

1 Introduction

Qanouni is a legal services mobile application that enables clients to submit legal requests to lawyers, follow up on their cases, chat securely, and schedule appointments. Lawyers can review incoming requests, accept or reject them, manage active cases, update case status, and receive ratings from clients after the service is delivered.

The main purpose of the project is to provide a centralized, digital channel for legal consultations instead of relying on scattered communication through phone calls, social media, or in-person visits. The target users are individual clients who need legal help and licensed lawyers or government legal offices who provide that help.

2 Problem Statement & Objectives

2.1 Problem Statement

Clients often struggle to find verified lawyers, submit legal documents in an organized way, and track the status of their cases. Communication is often informal and spread over multiple channels, which causes missed messages, lost files, and confusion about case progress and payments. There is also no simple way for clients to rate lawyers and build a transparent reputation system.

2.2 Project Objectives

The main objectives of Qanouni are:

- Provide a secure registration and login process for both clients and lawyers.
- Allow clients to submit legal requests with descriptions and file attachments.
- Enable lawyers to review and accept requests and automatically create cases.
- Support real-time chat between client and lawyer, including file sharing.
- Allow lawyers to manage schedules and appointments through a calendar view.
- Allow clients to rate lawyers after successful payment and case completion.

3 Stakeholders

3.1 Stakeholder Roles

- **Clients (Users):** Register in the app, submit legal requests, upload supporting documents, chat with lawyers, follow case status, and rate lawyers after the service.
- **Lawyers / Government Legal Offices:** Register as legal service providers, receive and review client requests, accept them, manage cases, update status, chat with clients, manage schedules, and receive ratings.

4 User Stories

4.1 List of User Stories

User stories are written in the format: *As a [user], I want to [action], so that [benefit].*

- As a client, I want to register using my email so that I can create an account and submit legal requests.
- As a lawyer, I want to register and wait for admin verification so that only verified lawyers can provide services.
- As a client, I want to submit a new legal request with files so that the lawyer can review all details of my case.
- As a lawyer, I want to accept a client request so that a formal case is created and tracked inside the system.
- As a client, I want to chat with my lawyer so that I can ask follow-up questions and send additional information.
- As a lawyer, I want to update the case status from the chat screen so that both parties know the current progress.
- As a client, I want to rate my lawyer after payment so that other clients can see the lawyer's reputation.
- As a lawyer, I want to add available schedule so I can manage my business.

5 Sprints

5.1 Sprint 1

- Component: Registration – Story: User Account Creation (US-01).
- Component: Login – Story: User Login (US-02).

5.1.1 Progress Summary

By the end of Sprint 1, the team implemented basic account creation and login using Firebase Authentication. Users could register with email, password, and profile information, then log in and be routed to the appropriate home screen based on their role (client or lawyer/government).

5.1.2 Issues Faced

- Handling different roles (client vs. lawyer/government) in the same registration and login flow.
- Deciding how to store the `isVerified` flag and when to allow the lawyer to enter the app.

5.2 Sprint 2

- Component: Lawyer and client Profile – Story: Lawyer and client Profile Creation (US-03).
- Component: Lawyer Search – Story: Find and Select a Lawyer (US-04).
- Component: Case Management – Story: Manage Cases and Track Status (US-05).
- Component: Document Upload – Story: Upload Case Documents (US-06).

5.2.1 Progress Summary

By the end of Sprint 2, lawyers could create and edit their profiles, and clients were able to search for lawyers and select one based on lawyer rating and speciality. The initial case management flow was implemented so that client requests created cases with statuses, and clients could upload case documents that were stored in Firebase Storage and linked to their requests.

5.2.2 Issues Faced

- Coordinating data between Firebase Realtime Database, Firestore conversations, and Storage paths for uploaded documents.

5.3 Sprint 3

- Component: Appointment Management – Story: Organize Appointments and Alerts (US-07).
- Component: Client Communication – Story: Lawyer and Client Communication (US-08).
- Component: Payment Management – Story: Handle Payments and Show Income (US-09).
- Component: Lawyer Evaluation – Story: Rate and Review Lawyer (US-10).

5.3.1 Progress Summary

Sprint 3 completed the main workflow: lawyers could manage appointment slots and schedules, real-time communication between client and lawyer was integrated with the related case ID ,payments were recorded per case, and clients could rate and review lawyers after successful payment.

5.3.2 Issues Faced

- Designing a consistent `conversationId` naming convention that works for all client–lawyer chat pairs.
- Designing the rating aggregation structure to compute average ratings efficiently and safely for each lawyer.

6 Functional Requirements

- The system shall allow users to register as clients or lawyers using email and password.
- The system shall require admin verification for lawyer and government accounts before login.
- The system shall authenticate users via Firebase Authentication.
- The system shall allow clients to submit legal requests that include description and file attachments.
- The system shall allow lawyers to view and accept pending client requests.
- The system shall create a new case when a lawyer accepts a request.
- The system shall allow both parties to chat inside each case.
- The system shall allow lawyers to update the status of a case.
- The system shall allow clients to rate the lawyer after payment.
- The system shall allow lawyers to create schedule entries (date, time, description) for appointments.

7 User Interface (UI) Design

7.1 Authentication Screen



Figure 1: Authentication Screen (Client / Lawyer)

Purpose

The authentication screen allows users to create a new account or log in using their email and password. It also lets the user select their role (Client or Lawyer / Government) before registration.

Key UI Elements

- App logo and branding.
- Role switch (Client vs Lawyer / Gov).
- Full Name, National ID, Phone fields (for registration).
- Email and Password fields.
- “Create Account” button.
- “Login” button.

7.2 Client home and notification

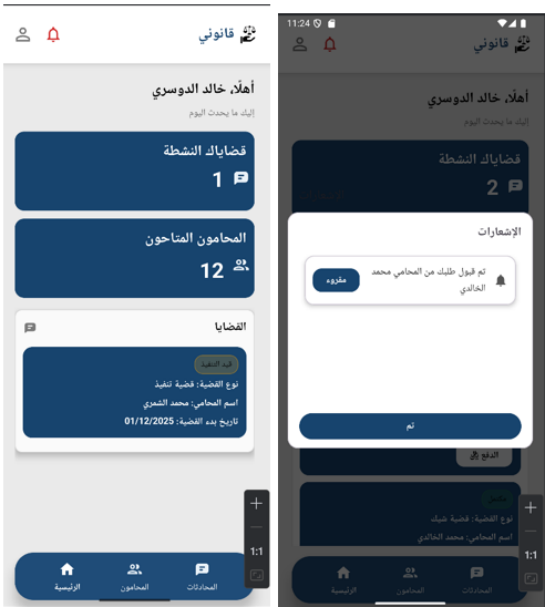


Figure 2: Client home screen (left) and notifications dialog (right):

Purpose

show the client his active cases and notifications for request decisions.

Key UI Elements

- Home cards for number of active cases and number of competing lawyers.
- Case card containing case type, lawyer name, start date, and status badge.
- Notification dialog listing messages such as “Your request was accepted” or “Your request was rejected” with a “Done” button.

7.3 Lawyer Schedule Screen

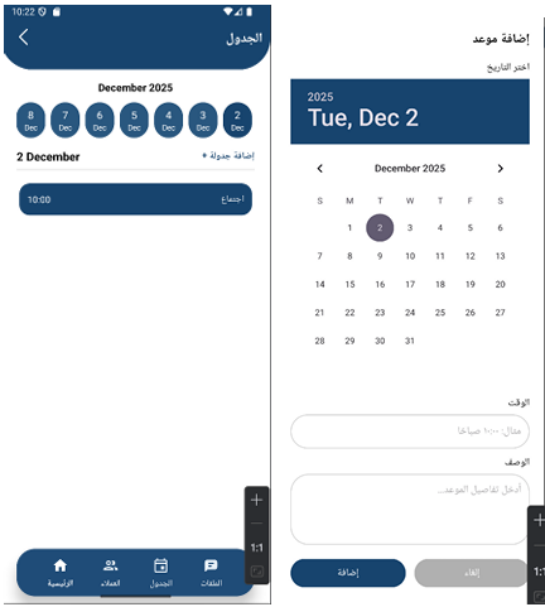


Figure 3: Lawyer schedule list (left) and add-appointment screen (right)

Purpose

The schedule screen allows the lawyer to manage daily appointment slots by selecting a date and adding time ranges and descriptions.

Key UI Elements

- Horizontal list of dates for the selected month.
- List of appointments for the selected date.
- "Add schedule" button that opens a full-screen date picker with time and description.

7.4 Lawyer Chat Screen



Figure 4: Lawyer chat screen with case status control and file messages

Purpose

The case chat screen shows messages between the client and the lawyer for a single case. It also displays the current case status and allows the lawyer to update it using a status spinner.

Key UI Elements

- Header containing client name, case identifier, and case status dropdown (Not Completed, In Progress, Com pleted).
- Text and file message bubbles for both users.
- Automatic system messages when the case status change.
- Message input area with attachment button and send button.

7.5 Lawyer Home Dashboard



Figure 5: Lawyer home dashboard with statistics and today's schedule.

Purpose

Provide the lawyer with an overview of today's work: number of clients, active cases, and total income, plus quick access to the schedule.

Key UI Elements

- Greeting message with the lawyer name and subtitle.
- Statistics cards: number of clients, number of active cases, and total income.
- "Today's Schedule" section with upcoming appointments.

7.6 Lawyer Requests and Existing Chats

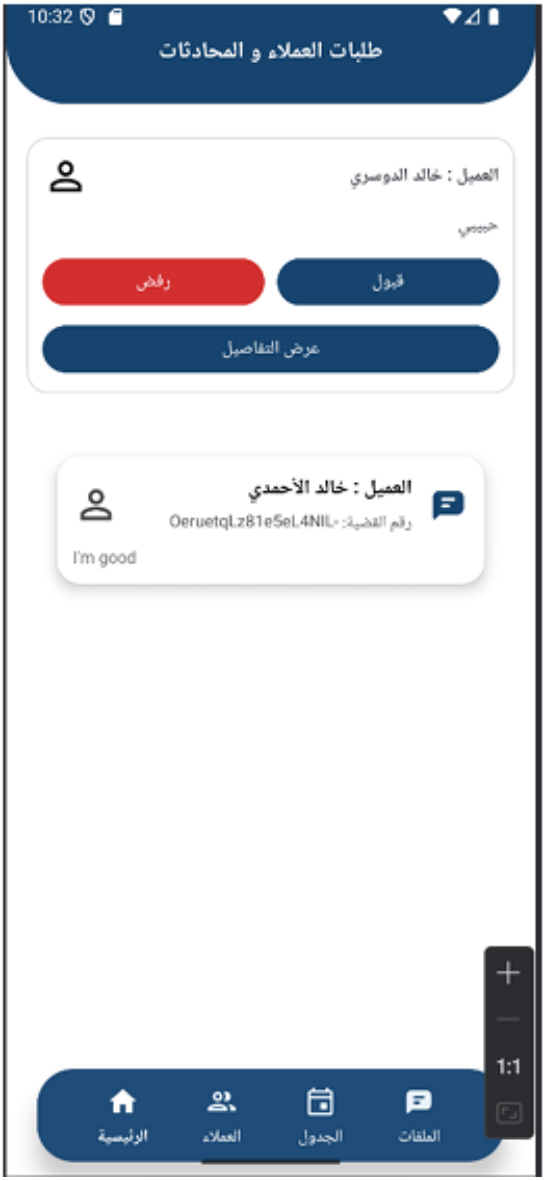


Figure 6: : Lawyer screen for client requests and existing chats.

Purpose

Show the lawyer all incoming client requests and existing chat conversations in one place.

Key UI Elements

- Request card with client name, short description, and caseID.
- Buttons to accept, reject, or view full details.
- List of existing chats with client name, last message, and time.

7.7 Lawyer Files and Profile Screens

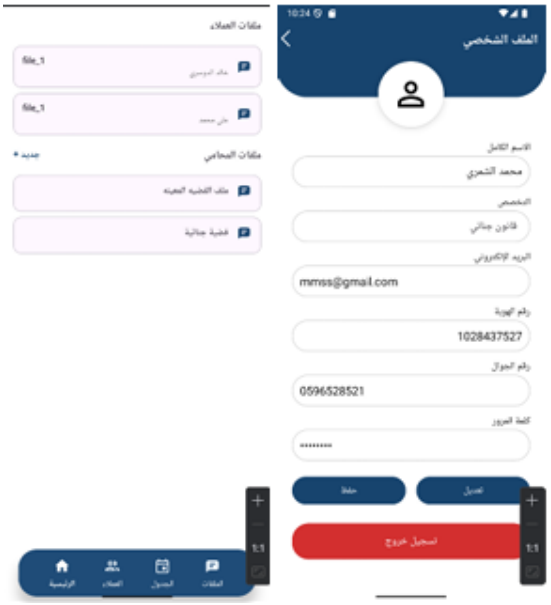


Figure 7: Lawyer files screen (left) and lawyer profile screen (right)

Purpose

manage client case files, private lawyer files, and the lawyer's personal profile.

Key UI Elements

- Files screen divided into "Client Files" and "Lawyer Files".
- File cards with name and quick actions (open, rename).
- Profile fields for name, specialization, email, national ID, phone number, and password.
- Buttons for "Edit", "Save", and "Logout".

7.8 Client Lawyer-Search and Request Dialog

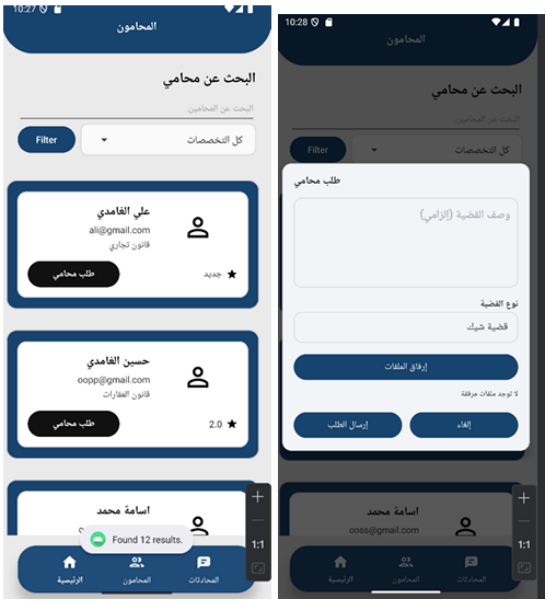


Figure 8: Client lawyer-search screen (left) and request dialog (right)

Purpose

Enable the client to search for lawyers by filters and submit a new case request to a selected lawyer.

Key UI Elements

- Search bar with filter button (e.g., specialization, rating).
- Lawyer cards with name, email, specialty, rating, and "Request lawyer" button.
- Request dialog with description text area, case-type spinner, "Attach files" button, and send / cancel buttons.

7.9 Client Conversations and Chat Screen

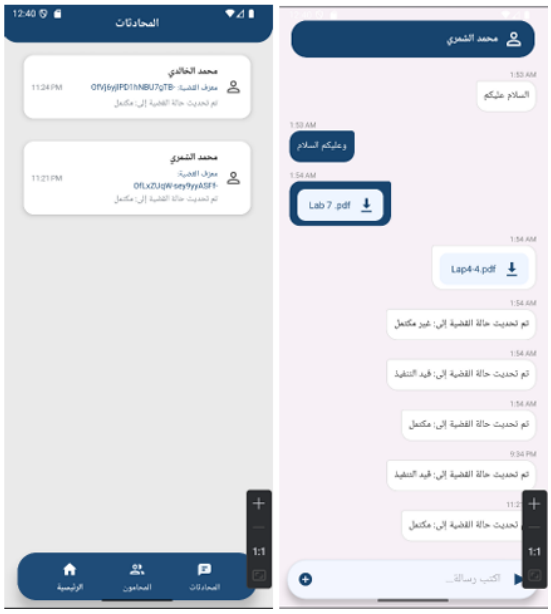


Figure 9: Client conversations list (left) and chat screen (right)

Purpose

Let the client see all conversations and chat with the assigned lawyer for each case.

Key UI Elements

- Conversations list with lawyer name, last message, and time.
- Chat screen with message bubbles, timestamps, and file messages.
- Input area with attachment button and send button.
- Automatic system messages when the case status changes.

7.10 Payment and Rating Screens

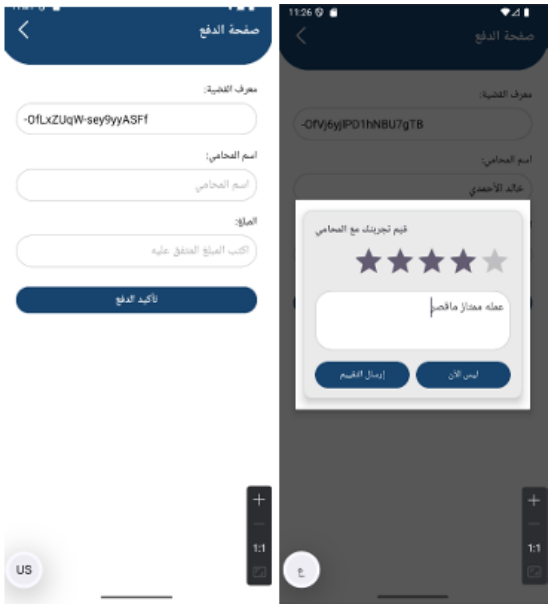


Figure 10: Payment screen (left) and rating dialog (right)

Purpose

Confirm the payment for a case and collect feedback from the client about the lawyer.

Key UI Elements

- Payment screen with case identifier, lawyer name, and amount.
- "Confirm payment" button.
- Rating dialog with star rating from 1 to 5 and notes text area.
- Buttons for "Submit rating" and "Not now".

8 Backend Logic & Functionality

8.1 Key Functions Overview

This section describes the core backend functions implemented in the Android application. Each function is shown as Java code and briefly explained.

8.1.1 `register()` – New User Registration

The `register()` method handles user registration for both clients and lawyers. It validates the input fields, creates an account in Firebase Authentication, stores the user profile in Realtime Database, sets the `isVerified` flag, and routes the user according to their role.

Listing 1: register() function

```
private void register() {
    final String name   = etFullName.getText().toString().trim();
    final String nid    = etNationalId.getText().toString().trim
       ();
    final String phone  = etPhone.getText().toString().trim();
    final String email  = etEmail.getText().toString().trim();
    final String pass   = etPassword.getText().toString();
    final String role   = rbLawyer.isChecked() ? "LAWYER_OR_GOV"
       : "USER";

    if (TextUtils.isEmpty(name) || TextUtils.isEmpty(nid) ||
            TextUtils.isEmpty(phone) || TextUtils.isEmpty(email)
               || TextUtils.isEmpty(pass)) {
        Toast.makeText(this, "                          .", Toast
           .LENGTH_SHORT).show();
        return;
    }
    if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        Toast.makeText(this, "
                    .", Toast.LENGTH_SHORT).show();
        return;
    }
    if (pass.length() < 6) {
        Toast.makeText(this, "                       6
                    .", Toast.LENGTH_SHORT).show();
        return;
    }

    btnCreateAccount.setEnabled(false);

    auth.createUserWithEmailAndPassword(email, pass)
            .addOnCompleteListener(this, t1 -> {
```

```
                if (!t1.isSuccessful()) {
                    btnCreateAccount.setEnabled(true);
                    Toast.makeText(RegisterActivity.this,
                            t1.getException()!=null? t1.
                               getException().getMessage():"
                                                          ",
                            Toast.LENGTH_SHORT).show();
                    return;
                }

                String uid = auth.getCurrentUser().getUid();
                User user = new User(name, nid, phone, email,
                   role, System.currentTimeMillis());

                usersRef.child(uid).setValue(user).
                   addOnCompleteListener(t2 -> {
                    if (!t2.isSuccessful()) {
                        btnCreateAccount.setEnabled(true);
                        Toast.makeText(RegisterActivity.this,
                                "

                                        .",
                                Toast.LENGTH_SHORT).show();
                        return;
                    }

                    boolean verified = "USER".equals(role);
                    usersRef.child(uid).child("isVerified").
                       setValue(verified)
                            .addOnCompleteListener(t3 -> {
                                btnCreateAccount.setEnabled(true)
                                   ;

                                if ("LAWYER_OR_GOV".equals(role))
                                    {
                                    Toast.makeText(
                                       RegisterActivity.this,
                                            "
                                                                .



                                                               .",
                                            Toast.LENGTH_LONG).
                                               show();
                                    FirebaseAuth.getInstance().
                                       signOut();
                                    Intent back = new Intent(
```

```
                                        RegisterActivity.this,
                                        MainActivity.class);
58                                  back.addFlags(Intent.
                                        FLAG_ACTIVITY_NEW_TASK |
59                                  Intent.FLAG_ACTIVITY_CLEAR_TASK)
                                        ;
60                                  startActivity(back);
61                              } else {
62                                  Intent i = new Intent(
                                        RegisterActivity.this,
                                        UserHomeActivity.class);
63                                  i.putExtra("name", name);
64                                  i.putExtra("role", role);
65                                  i.addFlags(Intent.
                                        FLAG_ACTIVITY_NEW_TASK |
66                                  Intent.FLAG_ACTIVITY_CLEAR_TASK
                                        );
67                                  startActivity(i);
68                              }
69                          });
70                  });
71              });
72  }
```

8.1.2 `login()` – User Login

The `login()` method validates the email and password, performs authentication using Firebase, and routes the user to the correct home screen.

Listing 2: login() function

```
1   private void login() {
2       String email = etEmail.getText().toString().trim();
3       String pass  = etPassword.getText().toString();
4
5       if (TextUtils.isEmpty(email) || TextUtils.isEmpty(pass)) {
6           Toast.makeText(this, "
                        .", Toast.LENGTH_SHORT).show();
7           return;
8       }
9       if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
10          Toast.makeText(this, "
                    .", Toast.LENGTH_SHORT).show();
11          return;
12      }
13
14      btnLogin.setEnabled(false);
```

```
15
16      auth.signInWithEmailAndPassword(email, pass)
17              .addOnCompleteListener(this, t -> {
18                  btnLogin.setEnabled(true);
19                  if (!t.isSuccessful()) {
20                      Toast.makeText(MainActivity.this,
21                              "
                                            .",
22                              Toast.LENGTH_SHORT).show();
23                      return;
24                  }
25                  routeToHome(auth.getCurrentUser().getUid());
26              });
27  }
```

8.1.3 `submitRequest()` – Client Submits Request

The `submitRequest()` method allows the client to send a new legal request to a selected lawyer, including file attachments uploaded to Firebase Storage.

Listing 3: submitRequest() function

```
1   private void submitRequest(String caseType) {
2
3       String lawyerId = getArguments() != null ?
4               getArguments().getString(ARG_LAWYER_ID) : null;
5       if (lawyerId == null) {
6           Toast.makeText(getContext(), "      :
                                    ",
7                       Toast.LENGTH_SHORT).show();
8           return;
9       }
10
11      String description = etDescription.getText().toString().trim
            ();
12      if (TextUtils.isEmpty(description)) {
13          etDescription.setError("                              ");
14          return;
15      }
16
17      String clientId = FirebaseAuth.getInstance().getUid();
18      if (clientId == null) {
19          Toast.makeText(getContext(), "
                        ", Toast.LENGTH_SHORT).show();
20          return;
21      }
22
```

```
    String requestId = UUID.randomUUID().toString();

    FirebaseStorage storage = FirebaseStorage.getInstance();
    StorageReference baseFolder = storage.getReference()
            .child("requests")
            .child(lawyerId)
            .child(requestId);

    List<Map<String, String>> filesMeta = new ArrayList<>();
    List<Task<Void>> uploadTasks = new ArrayList<>();

    for (int i = 0; i < selectedUris.size(); i++) {
        Uri uri = selectedUris.get(i);
        String name = "file_" + (i + 1);
        StorageReference fileRef = baseFolder.child(name);

        Task<Void> t = fileRef.putFile(uri)
                .continueWithTask(task -> fileRef.getDownloadUrl
                   ())
                .continueWith(task -> {
                    Uri downloadUri = task.getResult();
                    Map<String, String> m = new HashMap<>();
                    m.put("name", name);
                    m.put("url", downloadUri.toString());
                    m.put("storagePath", fileRef.getPath());
                    filesMeta.add(m);
                    return null;
                });

        uploadTasks.add(t);
    }

    Tasks.whenAll(uploadTasks)
            .addOnSuccessListener(v ->
                    writeRequestToDb(lawyerId, clientId,
                       requestId,
                                    description, filesMeta,
                                       caseType)
            )
            .addOnFailureListener(e ->
                    Toast.makeText(getContext(),
                            "                                  : " + e.
                               getMessage(),
                            Toast.LENGTH_LONG).show());
}
```

8.1.4 `acceptRequest()` – Lawyer Accepts Request

The `acceptRequest()` method is called by the lawyer to accept a client request. It creates a new case, mirrors it under the client node, updates the request status, stores a copy under `lawyerFiles`, updates the Firestore conversation with the `caseId`, and pushes a notification to the client.

Listing 4: acceptRequest() function

```
private void acceptRequest(String requestId, String clientId) {
    if (lawyerId == null) {
        Toast.makeText(this, "                    ", Toast.
           LENGTH_SHORT).show();
        return;
    }

    DatabaseReference src = db.child("requests").child(lawyerId).
       child(requestId);
    DatabaseReference dst = db.child("lawyerFiles").child(
       lawyerId)
            .child("clientFiles").child(clientId).child(requestId
               );

    DatabaseReference userCasesRef = db.child("userCases").child(
       clientId);

    src.get().addOnSuccessListener(snap -> {
        if (!snap.exists()) {
            Toast.makeText(this, "                              ",
               Toast.LENGTH_SHORT).show();
            return;
        }

        Map<String, Object> data = (Map<String, Object>) snap.
           getValue();
        if (data == null) data = new HashMap<>();
        data.put("acceptedAt", ServerValue.TIMESTAMP);
        data.put("status", "ACCEPTED");

        String caseType = snap.child("caseType").getValue(String.
           class);

        DatabaseReference newCaseRef = FirebaseDatabase.
           getInstance()
                .getReference("cases")
                .push();
        String caseId = newCaseRef.getKey();
        data.put("caseId", caseId);
```

```
31
32          Map<String, Object> caseData = new HashMap<>();
33          caseData.put("caseId", caseId);
34          caseData.put("lawyerId", lawyerId);
35          caseData.put("clientId", clientId);
36          caseData.put("requestId", requestId);
37          caseData.put("status", "                    ");
38          caseData.put("amount", 0);
39          caseData.put("createdAt", ServerValue.TIMESTAMP);
40          caseData.put("caseType", caseType);
41
42          Map<String, Object> finalData = data;
43          newCaseRef.setValue(caseData)
44                  .addOnSuccessListener(aVoid -> {
45                      userCasesRef.child(caseId).setValue(caseData)
                            ;
46
47                      src.child("status").setValue("ACCEPTED");
48                      src.child("caseId").setValue(caseId);
49                      dst.setValue(finalData);
50
51                      String convId = (lawyerId.compareTo(clientId)
                            < 0)
52                              ? lawyerId + "_" + clientId
53                              : clientId + "_" + lawyerId;
54
55                      FirebaseFirestore fs = FirebaseFirestore.
                            getInstance();
56
57                      Map<String, Object> caseMap = new HashMap<>()
                            ;
58                      caseMap.put("caseId", caseId);
59
60                      fs.collection("conversations").document(
                            convId)
61                              .set(caseMap, com.google.firebase.
                                    firestore.SetOptions.merge())
62                              .addOnFailureListener(e ->
63                                      Toast.makeText(this,
64                                              "
                                                    caseId
                                                                ",
65                                              Toast.LENGTH_SHORT).
                                                    show());
66
67                      Toast.makeText(this,
68                              "
```

```
                                        ",
                            Toast.LENGTH_SHORT).show();

                    FirebaseDatabase.getInstance().getReference("
                       notif")
                            .child(clientId)
                            .push()
                            .setValue(new java.util.HashMap<
                               String,Object>() {{
                                put("t", "
                                                        " +
                                   lawyerName);
                                put("ts", ServerValue.TIMESTAMP);
                                put("read", false);
                            }});
                })
                .addOnFailureListener(e ->
                        Toast.makeText(this,
                                "                              :
                                   " + e.getMessage(),
                                Toast.LENGTH_LONG).show());
    }).addOnFailureListener(e ->
            Toast.makeText(this,
                    "                    : " + e.getMessage(),
                    Toast.LENGTH_LONG).show());
}
```

8.1.5 `setupSpinner()` – Update Case Status from Chat

The `setupSpinner()` method configures the status spinner in the chat screen and updates the case status in Realtime Database when the lawyer selects a new value.

Listing 5: setupSpinner() function

```
private void setupSpinner() {
    ArrayAdapter<CharSequence> adapter = ArrayAdapter.
       createFromResource(
            this,
            R.array.case_status_list,
            android.R.layout.simple_spinner_item
    );
    adapter.setDropDownViewResource(android.R.layout.
       simple_spinner_dropdown_item);
    spCaseStatus.setAdapter(adapter);

    if (spCaseStatus.getSelectedItem() != null) {
```

```
        String status = spCaseStatus.getSelectedItem().toString();
        updateStatusChipBackground(status);
    }

    spCaseStatus.setOnItemSelectedListener(
        new android.widget.AdapterView.OnItemSelectedListener() {
        @Override
        public void onItemSelected(android.widget.AdapterView<?> parent,
                                   android.view.View view, int pos, long id) {

            if (isLoadingStatus) return;
            if (caseId == null || caseId.isEmpty()) return;

            String newStatus = parent.getItemAtPosition(pos).toString();
            if (newStatus.equals(caseStatus)) return;

            caseStatus = newStatus;

            DatabaseReference ref = FirebaseDatabase.getInstance()
                    .getReference("cases")
                    .child(caseId);

            ref.child("status").setValue(newStatus);
            ref.child("statusUpdatedAt").setValue(System.currentTimeMillis());

            sendStatusMessage(newStatus);
            updateStatusChipBackground(newStatus);
        }

        @Override
        public void onNothingSelected(
             android.widget.AdapterView<?> parent) { }
    });
}
```

8.1.6 `saveRating()` – Save Lawyer Rating

The `saveRating()` method stores the client rating for a case, sets a flag to prevent duplicate ratings, and updates the aggregate rating for the lawyer.

Listing 6: saveRating() function

```
private void saveRating(int stars, String notes, SimpleCb cb) {
    String clientId = FirebaseAuth.getInstance().getUid();
    if (clientId == null || lawyerId == null) {
        Toast.makeText(this, "                          ", Toast.
           LENGTH_SHORT).show();
        cb.done();
        return;
    }

    long now = System.currentTimeMillis();

    DatabaseReference ratingRef = FirebaseDatabase.getInstance()
            .getReference("ratings")
            .child(lawyerId)
            .child(caseId);

    Map<String, Object> rating = new HashMap<>();
    rating.put("stars", stars);
    rating.put("notes", notes);
    rating.put("clientId", clientId);
    rating.put("caseId", caseId);
    rating.put("createdAt", now);

    DatabaseReference ratedFlagRef = FirebaseDatabase.getInstance
       ()
            .getReference("cases")
            .child(caseId)
            .child("ratedByClient");

    DatabaseReference aggRef = FirebaseDatabase.getInstance()
            .getReference("users")
            .child(lawyerId)
            .child("ratingAggregate");

    ratingRef.setValue(rating).addOnSuccessListener(a -> {
        ratedFlagRef.setValue(true);

        aggRef.runTransaction(new Transaction.Handler() {
            @NonNull
            @Override
            public Transaction.Result doTransaction(@NonNull
               MutableData currentData) {
                Map<String, Object> agg =
                        (Map<String, Object>) currentData.
                           getValue();
```

```
                long count = 0, sum = 0;
                if (agg != null) {
                    Object c = agg.get("count");
                    Object s = agg.get("sum");
                    if (c instanceof Number) count = ((Number) c)
                       .longValue();
                    if (s instanceof Number) sum   = ((Number) s)
                       .longValue();
                }
                count += 1;
                sum   += stars;

                Map<String, Object> newAgg = new HashMap<>();
                newAgg.put("count", count);
                newAgg.put("sum",   sum);
                currentData.setValue(newAgg);

                return Transaction.success(currentData);
            }

            @Override
            public void onComplete(@Nullable DatabaseError error,
                                   boolean committed,
                                   @Nullable DataSnapshot
                                      currentData) {
                cb.done();
            }
        });
    }).addOnFailureListener(e -> {
        Toast.makeText(PaymentActivity.this,
                "                              : " + e.getMessage()
                   ,
                Toast.LENGTH_LONG).show();
        cb.done();
    });
}
```

8.1.7 `showAddScheduleDialog()` – Add New Schedule Slot

The `showAddScheduleDialog()` method opens a dialog that lets the lawyer select a date and enter a time and description, then saves the new schedule entry under the lawyer's schedule node.

Listing 7: showAddScheduleDialog() function

```
private void showAddScheduleDialog() {
    Dialog dialog = new Dialog(this);
```

```
    dialog.requestWindowFeature(Window.FEATURE_NO_TITLE);
    dialog.setContentView(R.layout.dialog_add_schedule);

    if (dialog.getWindow() != null) {
        dialog.getWindow().setLayout(
                WindowManager.LayoutParams.MATCH_PARENT,
                WindowManager.LayoutParams.WRAP_CONTENT
        );
    }

    DatePicker datePicker = dialog.findViewById(R.id.
       datePickerDialog);
    EditText etTime = dialog.findViewById(R.id.etTime);
    EditText etDescription = dialog.findViewById(R.id.
       etDescription);
    Button btnAdd = dialog.findViewById(R.id.btnAdd);
    Button btnCancel = dialog.findViewById(R.id.btnCancel);

    try {
        Date date = apiFormat.parse(selectedDate);
        if (date != null) {
            Calendar cal = Calendar.getInstance();
            cal.setTime(date);
            datePicker.updateDate(
                    cal.get(Calendar.YEAR),
                    cal.get(Calendar.MONTH),
                    cal.get(Calendar.DAY_OF_MONTH));
        }
    } catch (ParseException ignored) { }

    btnCancel.setOnClickListener(v -> dialog.dismiss());

    btnAdd.setOnClickListener(v -> {
        String time = etTime.getText().toString().trim();
        String description = etDescription.getText().toString().
           trim();

        Calendar cal = Calendar.getInstance();
        cal.set(datePicker.getYear(),
                datePicker.getMonth(),
                datePicker.getDayOfMonth());
        String dateKey = apiFormat.format(cal.getTime());

        if (time.isEmpty()) {
            Toast.makeText(this, "                    ", Toast.
               LENGTH_SHORT).show();
            return;
```

```
        }
        if (description.isEmpty()) {
            Toast.makeText(this, "                    ", Toast.
               LENGTH_SHORT).show();
            return;
        }

        DatabaseReference newRef = schedulesRef.push();
        ScheduleModel model = new ScheduleModel();
        model.setDate(dateKey);
        model.setTime(time);
        model.setDescription(description);

        newRef.setValue(model)
                .addOnSuccessListener(aVoid -> {
                    Toast.makeText(this,
                            "
                                    ",
                            Toast.LENGTH_SHORT).show();
                    dialog.dismiss();

                    if (dateKey.equals(selectedDate)) {
                        loadSchedulesForDate(selectedDate);
                    }
                })
                .addOnFailureListener(e ->
                        Toast.makeText(this,
                                "                  : " + e.
                                   getMessage(),
                                Toast.LENGTH_SHORT).show());
    });

    dialog.show();
}
```

9 Database Design

9.1 Collections / Nodes Overview

The application uses multiple Firebase services:

- **Authentication:** Stores user credentials (email/password).
- **Realtime Database:** Stores users, requests, cases, ratings, schedules, notifications, and other structured data.

- **Cloud Firestore:** Stores chat conversation metadata and is used to link chats to specific cases.
- **Firebase Storage:** Stores uploaded files for requests and chat attachments.

Main Realtime Database nodes:

- `users` – user profiles and rating aggregates.
- `requests` – pending requests per lawyer.
- `cases` – main case records.
- `userCases` – cases listed under each client.
- `lawyerFiles` – archived client files per lawyer.
- `ratings` – ratings per lawyer and case.
- `notif` – push-style notifications.
- `schedules` – schedule entries per lawyer.

Firestore collection:

- `conversations` – chat conversations between client and lawyer, including `caseId` and members.

9.2 Node: users

- `uid`: String
- `name`: String
- `nationalId`: String
- `phone`: String
- `email`: String
- `role`: String (`USER` or `LAWYER_OR_GOV`)
- `createdAt`: Number (timestamp)
- `isVerified`: Boolean
- `ratingAggregate`: Object { count, sum }

Example document:

```
"users": {
  "uid123": {
    "name": "Faisal Saud",
    "nationalId": "1234567890",
    "phone": "+9665XXXXXXXX",
    "email": "user@example.com",
    "role": "USER",
    "createdAt": 1730000000000,
    "isVerified": true,
    "ratingAggregate": {
      "count": 5,
      "sum": 23
    }
  }
}
```

10 Test Cases

Test Case ID	Description	Expected Result	Outcome
TC001	User taps on SIGN UP button on main screen.	User should be taken to the Registration screen.	Pass
TC002	User leaves all fields blank on Registration screen and taps SIGN UP button.	Message displayed: "Please Fill in all fields."	Pass
TC005	User enters First Name, Last Name, Email and Password then taps SIGN UP button.	All user information is saved.	Pass
TC001-L	User taps on Log IN button.	User should be taken to the home page.	Pass
TC002-L	User leaves fields blank on Login screen and taps Log in.	Message displayed: "Please Fill in all fields."	Pass
TC003-L	User enters invalid email.	Message: "Please enter Valid email." appears.	Pass
TC001-P	Lawyer taps the "Edit" button.	All input fields and specialty dropdown become editable.	Pass
TC002-P	User taps "Save" without pressing "Edit".	Toast appears: " ".	Pass
TC005-P	User saves changes after editing.	Data saved to Firebase, success toast displayed, fields disabled again.	Pass
TC001-A	Client opens "Available Lawyer" screen.	List of lawyers fetched and displayed from Firebase.	Pass
TC002-A	Client uses specialty filter dropdown.	Only lawyers matching selected specialty appear.	Pass
TC003-A	Client searches for lawyer by name.	Only lawyer matching entered name appears.	Pass

Table 1: Unified Test Cases for Qanouni

11 Challenges and Solutions

11.1 Challenges Faced

- **Multiple Firebase services:** Coordinating Realtime Database, Firestore, and Storage while keeping IDs and references consistent.
- **Role-based logic:** Handling different flows for clients and lawyers in the same app

without duplicating screens.

- **Case-linking in chat:** Ensuring that each conversation is linked to the correct `caseId` when a request is accepted.

11.2 Solutions Implemented

- Used a consistent ID strategy: `conversationId` is derived from the sorted pair of `clientId` and `lawyerId`, and `caseId` is pushed once and reused everywhere.
- Centralized role flags (`role`, `isVerified`) inside the user profile and routed users using a single `routeToHome()` method.
- Stored `caseId` inside the Firestore conversation document when a request is accepted, so any chat screen can directly retrieve the linked case.

12 Conclusion & Future Work

12.1 Conclusion

Qanouni provides a structured workflow for managing legal consultations between clients and lawyers. The project integrates authentication, request management, case creation, chat, scheduling, and rating into one coherent system. Through several sprints, the team implemented the core user stories and validated the main flows using test cases.

12.2 Future Work

- Add in-app payment integration and automatically link payments to case completion.
- Integrate an AI legal assistant (AI Agent) to help analyze uploaded documents, summarize case details, and provide suggestions to lawyers.
- AI-based smart matching system that automatically recommends the most suitable lawyer for a client based on case type, history, and lawyer specialization.
- Support multi-language UI and accessibility improvements.

13 Source Code

13.1 GitHub Link

Public repository (to be updated):
https://github.com/abdulamlik1122/QanuniApp

13.2 Figma Link

Public design prototype (to be updated):
https://www.figma.com/design/uRkoyfozwvfTlUesV01nFb/Qanuni-App?version-id=22867474912298
node-id=0-1&p=f&t=snEXHBBjhKo8uh8m-0